

UI-TARS: 开创纯截图驱动的原生 GUI 操作智能体

Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, 等
ByteDance Seed (字节跳动) / Tsinghua University (清华大学)

英文原标题: UI-TARS: Pioneering Automated GUI Interaction with Native Agents

发表: arXiv 2501.12326 (2025 年 1 月, 字节跳动 ByteDance Seed) 引用数: 449 (Semantic Scholar, 2026-06-07)

代码: <https://github.com/bytedance/UI-TARS>

一句话定位: 做一个“只看屏幕截图就能像人一样操作电脑/手机”的端到端原生 GUI 智能体——不读 HTML、不用辅助接口, 看界面 → 想清楚 → 点哪/输什么, 全部由一个模型搞定, 并能从自己的操作历史里反思改进。

摘要

本文介绍 UI-TARS, 一个原生 GUI 智能体模型 (native GUI agent model), 它仅以屏幕截图作为输入, 并执行类人交互 (如键盘、鼠标操作)。与当前主流的、依赖被层层包装的商业模型 (如 GPT-4o) 外加专家精心设计的提示词和工作流的智能体框架不同, UI-TARS 是一个端到端模型, 性能却超过了这些复杂框架。实验表明其性能领先: UI-TARS 在 10 多个评测感知、定位 (grounding) 与 GUI 任务执行能力的基准上取得 SOTA。尤其在 OSWorld 基准上, UI-TARS 在 50 步预算下得分 24.6、15 步下得分 22.7, 分别超过 Claude 的 22.0 和 14.9; 在 AndroidWorld 上得分 46.6, 超过 GPT-4o 的 34.5。UI-TARS 包含若干关键创新: (1) **增强感知**: 利用大规模 GUI 截图数据集, 实现对 UI 元素的上下文感知理解与精确描述; (2) **统一动作建模**: 把动作标准化到一个跨平台统一的动作空间, 并通过大规模动作轨迹实现精确定位与交互; (3) **System-2 推理**: 把审慎推理融入多步决策, 涵盖任务分解、反思思考、里程碑识别等多种推理模式; (4) **基于反思式在线轨迹的迭代训练**: 通过在数百台虚拟机上自动采集、筛选并反思式精炼新的交互轨迹来突破数据瓶颈。通过迭代训练与反思微调, UI-TARS 能持续从自身错误中学习, 并在极少人工干预下适应未曾见过的情况。我们还分析了 GUI 智能体的演进路径, 以指导该领域的进一步发展。UI-TARS 已在 <https://github.com/bytedance/UI-TARS> 开源。

1 小白知识卡: 读这篇前要懂的背景

GUI 智能体 (GUI Agent)

能自动操作图形界面 (按钮、输入框、菜单……) 来完成任务的 AI, 比如让它“帮我订一张明天去北京的机票”, 它自己点来点去把任务做完。

原生智能体模型 vs. 智能体框架 (Native Agent Model vs. Agent Framework)

框架 = 把一个强模型（如 GPT-4o）当大脑，外面再用人手写的提示词、脚本、各种工具模块（看图的、记忆的、规划的）拼起来——**设计驱动**，环境一变就容易崩。**原生模型** = 把所有能力（看、想、记、做）都学进同一个模型的参数里，端到端，**数据驱动**，能持续学习适应。本文主张后者是未来。

纯视觉 / 截图驱动 (Pure-Vision)

模型只看界面截图（像人用眼睛看屏幕），不去读网页的 HTML 或系统的无障碍树（accessibility tree）。好处：不用系统权限、跨平台通用、更接近人的认知方式。

定位 / 接地 (Grounding)

把“我想点的那个东西”精确翻译成屏幕上的坐标 (x, y)。比如“点右上角那个写着 Submit 的红按钮” → 输出像素坐标。这是截图智能体能不能真正“点对地方”的关键。

System-1 与 System-2 思维 (快思考与慢思考)

借自心理学：**System-1** = 快、直觉、不假思索（如熟练地点个常见按钮）；**System-2** = 慢、刻意、一步步分析（如规划整个流程、出错后反思纠错）。本文重点是给模型装上 System-2：动手前先生成一段“思考 (thought)”。

VLM (视觉语言模型, Vision-Language Model)

能同时看图又能读写文字的大模型。本文的底座是 **Qwen-2-VL** (7B 和 72B 两个规模)。

行为克隆 (Behavior Cloning)

最朴素的“照着人怎么做来学”：给模型看“专家在某界面做了什么动作”，让它模仿。本文用它来学习多步操作轨迹。

SFT / DPO (监督微调 / 直接偏好优化, Direct Preference Optimization)

SFT = 拿“正确示范”让模型照着学。**DPO** = 给模型成对的“好动作 vs. 坏动作”，直接告诉它“这个比那个好”，让它更偏向好动作、远离坏动作。本文用 DPO 把“犯错的步骤”也利用起来（SFT 只用对的步骤，浪费了错的）。

推理时扩展 / Best-of-N (Inference-time Scaling)

不改模型参数，靠“多采样几次再挑最好的”来提升表现。本文实验里让模型对同一输入采样 $N = 1/16/64$ 个候选，发现 System-2 在 N 大时优势明显。

2 逐节精读

2.1 引言 (§1)：为什么要做原生 GUI 智能体

论文先点出大方向：自主智能体 (autonomous agents) 要能在极少人工监督下感知环境、决策、执行动作。而让智能体**无缝操作图形界面 (GUI)** 是一个关键前沿——因为人类的数字工作充满了按

钮、文本框、图片。

过去的两大老路子及其毛病：

- **依赖文本表示**（读 HTML、DOM、无障碍树）：平台之间不一致、太啰嗦、可扩展性差，还经常需要系统级权限才能拿到这些信息，限制了通用性。
- **模块化的智能体框架**：把功能拆成多个组件（用 GPT-4o 做理解推理、另用工具做定位或记忆），靠人工手工拼装。开发某类任务快，但依赖专家知识与任务定制，**换个陌生任务或环境一变就容易失败**。

两个关键转向（这就是“原生”的含义）：（1）从依赖文本 → **纯视觉**（只看截图）；（2）从模块化框架 → **端到端模型**。论文有一句很点题的对比（关键句翻译）：

“框架是**设计驱动的**，需要大量人工工程和预定义 workflow 来维持稳定、防止意外；而智能体模型本质是**数据驱动的**，能够通过大规模数据和迭代反馈来学习和适应。”

但原生智能体目前为何还不够好？两大根源：（1）GUI 领域本身难——感知上要理解高信息密度且不断变化的界面；推理规划要能导航和操纵界面；还要用到记忆（参考过去交互）；更要执行**精确的底层动作**（输出准确的点击坐标、往正确的输入框打字）。（2）从框架转向模型带来**根本性的数据瓶颈**：模块化框架可以用各组件单独的数据集（好凑），但端到端模型需要**把感知、推理、记忆、动作整合在一个统一 workflow 里的数据**，而这种富含人类专家 workflow 知识的数据历史上极其稀缺。

本文的四大核心贡献（对应后文 §4.2–4.5）：

1. **增强 GUI 截图感知** (§4.2)：用专门的解析工具采集网站/应用/系统的截图与元数据，构建大规模数据集，设计五类任务（元素描述、密集描述、状态转移描述、问答、Set-of-Mark 标注提示）。
2. **统一动作建模与多步执行** (§4.3)：设计跨平台统一动作空间；用标注轨迹 + 标准化的开源数据训练多步执行；用“元素描述配坐标”的大数据提升定位能力。
3. **面向审慎决策的 System-2 推理** (§4.4)：爬取并精炼约 **600 万条** GUI 教程注入推理知识；给所有动作轨迹增补“**思考 (thought)**”，注入任务分解、长期一致性、里程碑识别、试错、反思等推理模式——**每个动作前先生成显式思考**。
4. **从经验学习的迭代精炼** (§4.5)：在数百台虚拟机上在线探索、采集、筛选、反思式精炼新轨迹回灌模型，持续迭代；并通过**反思微调**和 **Agent DPO** 让模型学会识别并从错误中恢复。

底座是 **Qwen-2-VL** (7B 与 72B)，在约 **500 亿 (50B) token** 上继续训练，得到 UI-TARS-7B 与 UI-TARS-72B。（注：模型名 TARS 致敬电影《星际穿越》里那个机智、善于解决问题的机器人 TARS。）

2.2 GUI 智能体的演进路径 (§2)

论文把 GUI 智能体的发展按“需要多少人工干预、泛化能力多强”分为四个阶段（见图 1 的脉络，原文为 Figure 2/3）：

- **阶段 1：基于规则的智能体 (Rule-based)**。如 RPA（机器人流程自动化）系统：把用户指令匹配到预定义规则、调用 API。做固定重复任务有效，但完全靠人定规则、需要直接的 API/系统权限，遇到新情况就抓瞎，无法从环境或经验中学习。
- **阶段 2：智能体框架 (Agent Framework)**。用 GPT-4 / GPT-4o 这类基础模型当大脑 (AutoGPT、LangChain、ReAct、SeeAct 等)，靠提示工程与多步推理变灵活。但“ **workflows 知识**”仍是**人手工编码**的（提示词、外部脚本、工具使用启发式），带来三大弊端：**脆弱且维护成本高**（任务/界面一变就得重写规则）、**学习范式割裂**（很少用新经验去更新底层模型参数）、**模块不兼容**（多个模块靠提示或胶水代码协调，一处出错整条链崩，还得专家来调试）。
- **阶段 3：原生智能体模型 (Native Agent Model)** —— **本文所处的阶段**。把 workflows 知识直接**学进模型参数**，端到端统一感知/推理/记忆/动作，数据驱动。优势：整体学习与适应、减少人工程、统一参数带来强泛化、天然支持在线/终身学习。代表工作：Claude Computer-Use、Aguvis、ShowUI、OS-Atlas 等。
- **阶段 4：主动与终身学习智能体 (Active & Lifelong, 展望)**。智能体自己提出任务、执行、评估结果、自我打分奖励，靠自驱探索发现新知识、持续提升，几乎不依赖人工标注。与阶段 3 的本质区别在于**学习过程的自主性**：原生模型仍依赖人，主动智能体自己驱动学习。本文目标是建一个可扩展、数据驱动的原生模型，为这一阶段铺路。

2.3 原生智能体模型的核心能力 (§3)

论文把框架时代的模块“内化”为模型的四大核心能力：

感知 (Perception)。按输入特征把已有工作分三类：**结构化文本** (HTML/DOM/无障碍树，早期 LLM 智能体被迫这么做)、**视觉截图** (用 SoM 提示、OCR、GUI 元素检测如 DINO 来理解界面)、**综合界面建模** (同时用文本、视觉、语义轮廓)。还强调**实时感知**的重要性（界面是动态的，比如出现加载转圈要识别为“正在处理”）。结论：原生模型应像人一样**直接通过视觉感知**并把动作精确接地到原始截图上。

动作 (Action)。要点：**统一且多样的动作空间** (把各平台的 click/type/scroll/drag 抽象成一套通用操作，区分原子动作与组合动作)；**坐标定位的挑战** (不同设备宽高比、动态内容让精确定位很难)。

推理 (System-1 & System-2)。System-1 快而直觉但只能反应式一步决策；System-2 显式生成中间思考 (用 CoT 或 ReAct)，包含五个要素：**任务分解、长期一致性、里程碑识别、试错、反思**。UI-TARS 重点强化 System-2。

记忆 (Memory)。分**短期记忆**（当前任务的动作历史、当前状态、执行轨迹）与**长期记忆**（过往任务的执行路径、背景知识）。原生模型把长期经验**编码进自身参数**，用 In-Context Learning 或 CoT 来激活。

能力评测分类 (§3.2)：感知评测、定位评测，以及智能体能力评测——又分**离线**（静态预定义环境，评单步，用 step-level 指标如 Action-Matching Score）与**在线**（可交互的动态环境，评能否真正完成整个任务，用 task-level 成功率）。

2.4 UI-TARS 总体架构 (§4.1)

这是方法部分的总入口，对应图 1（原文 Figure 4）。给定一个初始任务指令，UI-TARS 不断地从设备收到**观测 (observation, 即截图)**、执行对应**动作 (action)**。整个过程可形式化为一个交替序列：

$$(\text{instruction}, (o_1, a_1), (o_2, a_2), \dots, (o_n, a_n)),$$

其中 o_i 是第 i 步的观测（设备截图）， a_i 是智能体执行的动作。每一步，模型输入**任务指令 + 历史交互** $(o_1, a_1, \dots, o_{i-1}, a_{i-1})$ + **当前观测** o_i ，输出一个来自预定义动作空间的动作 a_i ；执行后设备给出下一帧观测，循环往复。

为强化推理、促成更审慎的决策，作者在**每个动作 a_i 之前插入一段“思考 (thought)” t_i** ，体现 System-2 的反思特性。它是承上启下的关键中间步骤，让智能体在前进前先重新审视之前的动作和观测。这受 ReAct 启发，但比 ReAct 更结构化、更目标导向。于是过程变为：

$$(\text{instruction}, (o_1, t_1, a_1), (o_2, t_2, a_2), \dots, (o_n, t_n, a_n)).$$

为了在有限 token 预算内（如 32k 序列长度）控制开销，输入只保留**最近 N 帧观测**，但完整的历史动作与思考作为短期记忆保留。模型迭代预测当前思考 t_n 和动作 a_n ：

$$P(t_n, a_n \mid \text{instruction}, t_1, a_1, \dots, (o_{n-i}, t_{n-i}, a_{n-i})_{i=1}^N, o_n).$$

公式拆解：竖线 | 左边是**要预测的**（这一步的思考 + 动作），右边是**已知条件**（任务指令、全部历史思考与动作、最近 N 帧观测、当前帧 o_n ）。实验中 N 取 5。

2.5 增强 GUI 感知 (§4.2)

两大难点：(1) **截图稀缺**——通用自然图像很多，GUI 截图相对少；(2) **信息密度高、精度要求高**——一张界面常有上百个元素、布局复杂，且很多元素极小（如 1920×1080 图里 10×10 像素的图标），难以准确感知和定位。

数据采集：用专门解析工具，一边渲染截图一边自动抽取丰富元数据（元素类型、层级深度、包围框 bounding box、文本内容），结合自动爬取 + 人工辅助探索，覆盖主界面与深层嵌套页面，统一记成（截图，元素框，元素元数据）结构。采用**自底向上**的构建思路：先搞懂局部小元素，再整合到整体界面理解。

基于这些数据，设计了**五类核心任务**（对应原文 Figure 5）：

1. **元素描述 (Element Description)**: 对每个元素给出结构化描述, 含元素类型、视觉外观、位置信息、元素功能; 训练模型枚举截图里所有可见元素并描述。
2. **密集描述 (Dense Captioning)**: 对整张界面做全面细致的描述, 不仅是元素本身, 还包括空间关系与整体布局 (嵌入图片也生成描述), 同时尽量减少幻觉。
3. **状态转移描述 (State Transition Captioning)**: 识别并描述**两张连续截图之间的差异**, 判断是否发生了某个动作 (如鼠标点击、键盘输入), 也包含非交互式变化 (动画、刷新)。这对细粒度交互理解、动态状态感知很关键。
4. **问答 (QA)**: 合成涵盖界面理解、图像解读、元素识别、关系推理的多样问答, 提升抽象与推理能力。
5. **Set-of-Mark (SoM) 提示**: 按坐标在截图上画出形状/颜色/大小各异的视觉标记, 让模型更好地把视觉标记与对应元素关联起来。

2.6 统一动作建模与定位 (§4.3)

核心方法是**行为克隆**: 让模型模仿人类执行任务的动作。但真实任务是**多步序列**, 所以必须在多步轨迹上训练 (这对应 System-1 的“怎么把动作连起来”)。

统一动作空间 (Unified Action Space, 见表 1): 把各平台语义等价的动作标准化, 比如 Windows 上的“click”与手机上的“tap”统一, 便于跨平台知识迁移; 同时为每个平台保留专属可选动作。还定义两个**终止动作**: `Finished()` 表示任务完成, `CallUser()` 表示需要用户介入 (如登录、验证)。

动作轨迹采集: 两个来源——(1) **自标注数据**: 自研标注工具捕获用户在 PC 上各类软件/网站的操作, 先拟定任务指令、标注者审校后执行、再严格质量过滤; (2) **开源数据**: 整合 MM-Mind2Web、GUIAct、AITW、AITZ、AndroidControl、GUI-Odyssey、AMEX 等, 统一成同一动作空间格式。统计见表 2。

提升定位能力: 定位数据比多步数据**更容易规模化** (主要靠元素的视觉与位置属性, 可合成或抽取)。训练模型**直接预测要交互元素的坐标**: 对每个有包围框的元素, 取四角平均得到中心点; 输出**归一化到屏幕尺寸的相对坐标** (保证不同分辨率设备一致)。例如给描述“右上角写着 Submit 的红按钮”, 模型预测该按钮的归一化坐标。还整合了 SeeClick、MultiUI、Rico-SCA、CLAY、OmniACT、OS-ATLAS 等大量开源定位数据。

表 1: 统一动作空间（节选，对应原文 Table 1）。Shared 为跨平台共享动作，Desktop / Mobile 为各平台专属。

平台	动作	含义
Shared（共享）	Click(x, y)	在坐标 (x, y) 处点击
	Drag(x1, y1, x2, y2)	从 (x1, y1) 拖拽到 (x2, y2)
	Scroll(x, y, direction)	在 (x, y) 处按方向滚动
	Type(content)	输入指定内容
	Wait()	短暂等待
	Finished()	标记任务完成（终止）
	CallUser()	请求用户介入（终止）
Desktop（桌面）	Hotkey(key)	按下指定快捷键
	LeftDouble(x, y)	在 (x, y) 双击
	RightSingle(x, y)	在 (x, y) 右键单击
Mobile（移动）	LongPress(x, y)	在 (x, y) 长按
	PressBack()	按“返回”键
	PressHome()	按“主页”键
	PressEnter()	按“回车”键

表 2: 定位数据与多步动作轨迹数据的基本统计（对应原文 Table 2），对比作者自标注数据与开源数据。Ele.= 元素数，Trace= 动作轨迹数，avg steps= 平均步数。“*”表示作者未披露具体数值。

数据类型	平台	Ele.（元素）	Ele./Image	Trace（轨迹）	avg steps
Open Source（开源）	Web	14.8M	6.7	6.4k	7.1
	Mobile	2.5M	4.6	145k	9.6
	Desktop	1.1M	6.2	0	0
Ours（本文标注）	—	*	7.5	*	14.9

2.7 注入 System-2 推理 (§4.4)

只靠 System-1 直觉无法应对复杂多变的场景，因此要让模型学会**通过理解任务的全局结构来灵活规划动作步骤**。分两步走：

第一步：用 GUI 教程做推理富化 (Reasoning Enrichment)。网上图文交织的教程天然演示了各种软件/网页的详细交互，是建立 GUI 知识、引入逻辑推理模式的理想来源。作者从 MINT 和 OmniCorpus 两个图文交织数据集出发，做多阶段过滤：(1) **粗过滤**——训练一个 fastText 分类器筛出“像教程”的内容；(2) **细过滤**——用 LLM 剔除误报；(3) **去重与精炼**——用 URL 和 LSH（局部敏感哈希）去重，再让 LLM 改写文本。最终得到约 **600 万**条高质量 GUI 教程，平均每条 510 个文本 token、3.3 张图。

第二步：用“思考增补”做推理激发 (Thought Augmentation)。§4.3 采集的轨迹只有“观测-动

作”、**缺少显式思考**。于是给数据增补思考 t ，把格式从 $(o_{i-1}, a_{i-1}, o_i, a_i, \dots)$ 变成 $(o_{i-1}, t_{i-1}, a_{i-1}, o_i, t_i, a_i, \dots)$ 。两种标注方式：

(1) *ActRe* (反向标注)：对每个轨迹按步拆分，对第 n 步，用 VLM 在“已知前文上下文 + 当前目标动作 a_n ”的条件下**反推出**思考 t_n ：

$$t_n = \text{VLM}(\text{instruction}, (o_1, t_1, a_1), (o_2, t_2, a_2), \dots, o_n, a_n)$$

$$t_{n+1} = \text{VLM}(\text{instruction}, (o_1, t_1, a_1), \dots, (o_n, t_n, a_n), o_{n+1}, a_{n+1})$$

这样生成的思考在逻辑上扎根于前文、且与当前动作对齐。标注时引导 VLM 表现出 System-2 的五种推理模式（对应原文 Figure 6）：**任务分解、长期一致性、里程碑识别、试错、反思**。

(2) 思考自举 (*Thought Bootstrapping*, 正向标注)：上面的反向标注有个隐患——**先知道答案再编理由**，生成的思考可能只是“表面上”对得上动作，并非真正的因果推理（假阳性）。为此改用**不预先知道正确动作**的自举方式：让早期检查点模型 UI-TARS_{early} 采样多个“思考-动作”对，再**挑出那个能导出正确动作的思考**：

$$(\hat{t}_n^i, \hat{a}_n^i)_{i=1}^{\text{max-try}} = \text{UI-TARS}_{\text{early}}(\text{instruction}, (o_1, t_1, a_1), \dots, o_n)$$

$$\text{Select}(\hat{t}_n^i, \hat{a}_n^i), \text{使得 } \hat{a}_n^i = a_n$$

这迫使模型**真正模拟决策过程**而非给既定动作找借口，标注质量更高。思考用**中英双语**标注以增加语言多样性；同时训练里也保留不带思考的原始轨迹。

2.8 从长期记忆中的先验经验学习 (§4.5)

GUI 智能体难以像 LLM 那样规模化，根因是**大规模、标准化、真实世界的 GUI 操作过程数据极其稀缺**。解法：从长期记忆里的先验经验学习——让 UI-TARS 在**真实设备上动态学习**，通过半自动的采集-过滤-精炼持续自我提升。

在线轨迹自举 (Online Trace Bootstrapping, 对应原文 Figure 7)：先拿到多样的任务目标（人工标注 + 模型生成）。在第 n 轮，智能体 M_n 在目标 GUI 环境（如虚拟 PC）里执行指令集 I_n ，产生原始轨迹集 $T_{\text{raw},n}$ 。再用**多级过滤**得到干净轨迹：

$$\text{Filter}(T_{\text{raw},n}, I_n) = T_{\text{filtered},n},$$

过滤含三步：(1) **规则奖励**——启发式规则去掉明显异常（如不改变环境的冗余动作）；(2) **VLM 打分**——低于阈值的轨迹剔除；(3) **人工复核**——标注者找出出错步、丢弃其后动作、只保留有效前缀。然后用过滤后的轨迹自我提升、并扩充指令集：

$$M_{n+1} = \text{FineTune}(M_n, T_{\text{filtered},n}), \quad I_{n+1} = \text{HumanRefine}(I_n, T_{\text{filtered},n}).$$

在数百台虚拟 PC 上多轮迭代，不断用最新模型生成新轨迹。

反思微调 (Reflection Tuning)：真实在线部署中，智能体常因不会自我反思而**卡死**（如反复点一个没反应的按钮）。而离线数据大多是“理想化、零错误”的轨迹，反而**教不会模型从错误中恢复**。

为此提出反思微调：把模型自己犯的真实错误及其修正暴露给它学习。对一条出错的在线轨迹，设第 τ 步动作 a_τ 无效或次优，标注者给出修正后的思考与动作 t_τ^*, a_τ^* ，得到**错误纠正对**：

$$\begin{aligned} T_- &= (\text{instruction}, (o_1, t_1, a_1), \dots, (o_\tau, t_\tau, a_\tau)) \\ T_+ &= (\text{instruction}, (o_1, t_1, a_1), \dots, (o_\tau, t_\tau^*, a_\tau^*)) \end{aligned}$$

更巧妙的是，还要求标注者**基于错误动作 a_τ 继续标注下一步**（模拟“错误已经发生”的情形）：下一步的思考 $t_{\tau+1}^*$ 必须**承认上一步失误、弥补其影响**，再给出能让任务重回正轨的 $a_{\tau+1}^*$ （例如：上一步想加书签却误点了关闭，下一步就该重新打开刚关掉的网页再去点书签）。这构成**反思后 (post-reflection) 对**。训练时只用正样本 T_+ 做 SFT，且**只对修正步计损失**，错误步不参与训练。

Agent DPO：在线自举会自然产生大量**错误步骤（负样本）**，但 SFT 只用了修正的正样本、浪费了负样本。于是引入 **DPO**，同时利用修正动作与错误动作。设状态 s_τ 包含指令与到当前步的交互历史，原本错误动作 a_τ 被修正为偏好动作 a'_τ 。基于 Bradley-Terry 模型，成对偏好似然为：

$$P_\theta(a'_\tau \succ a_\tau \mid s_\tau) = \frac{\exp(r_\theta(s_\tau, a'_\tau))}{\exp(r_\theta(s_\tau, a_\tau)) + \exp(r_\theta(s_\tau, a'_\tau))},$$

其中 $r_\theta(s, a)$ 是学到的奖励函数， $a'_\tau \succ a_\tau$ 表示 a'_τ 比 a_τ 更受偏好。沿用 DPO 的推导（带 KL 约束的 RL 目标下的解析最优策略），直接在偏好数据上优化：

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_\tau \left[\log \sigma \left(\beta \log \frac{\pi_\theta(a'_\tau | s_\tau)}{\pi_{\text{SFT}}(a'_\tau | s_\tau)} - \beta \log \frac{\pi_\theta(a_\tau | s_\tau)}{\pi_{\text{SFT}}(a_\tau | s_\tau)} \right) \right].$$

公式拆解： π_θ 是待优化的智能体， π_{SFT} 是 SFT 后的参考智能体， β 控制二者偏离程度， σ 是 sigmoid。直觉上：**提高修正动作 a'_τ 的概率、压低错误动作 a_τ 的概率**，从而隐式地用一个奖励函数把模型推向更好的决策。

2.9 训练 (§4.6)：三阶段流程

为与 Aguis、OS-Atlas 公平对比，沿用同样的 VLM 底座 **Qwen-2-VL**，总数据量约 **50B token**，分三阶段、逐步引入更高质量数据：

1. **继续预训练 (Continual Pre-training)**：用 §4 的全部数据（不含反思微调数据），恒定学习率，打牢感知/定位/动作轨迹的基础。
2. **退火 (Annealing)**：选取感知、定位、动作轨迹、反思微调数据的高量子集做退火，逐步收敛学习动态，得到 **UI-TARS-SFT**。
3. **DPO 阶段**：用在线自举得到的反思配对数据做 DPO，强化最优动作、惩罚次优动作，得到最终的 **UI-TARS-DPO**。

3 方法图解：UI-TARS 整体架构（图 4）

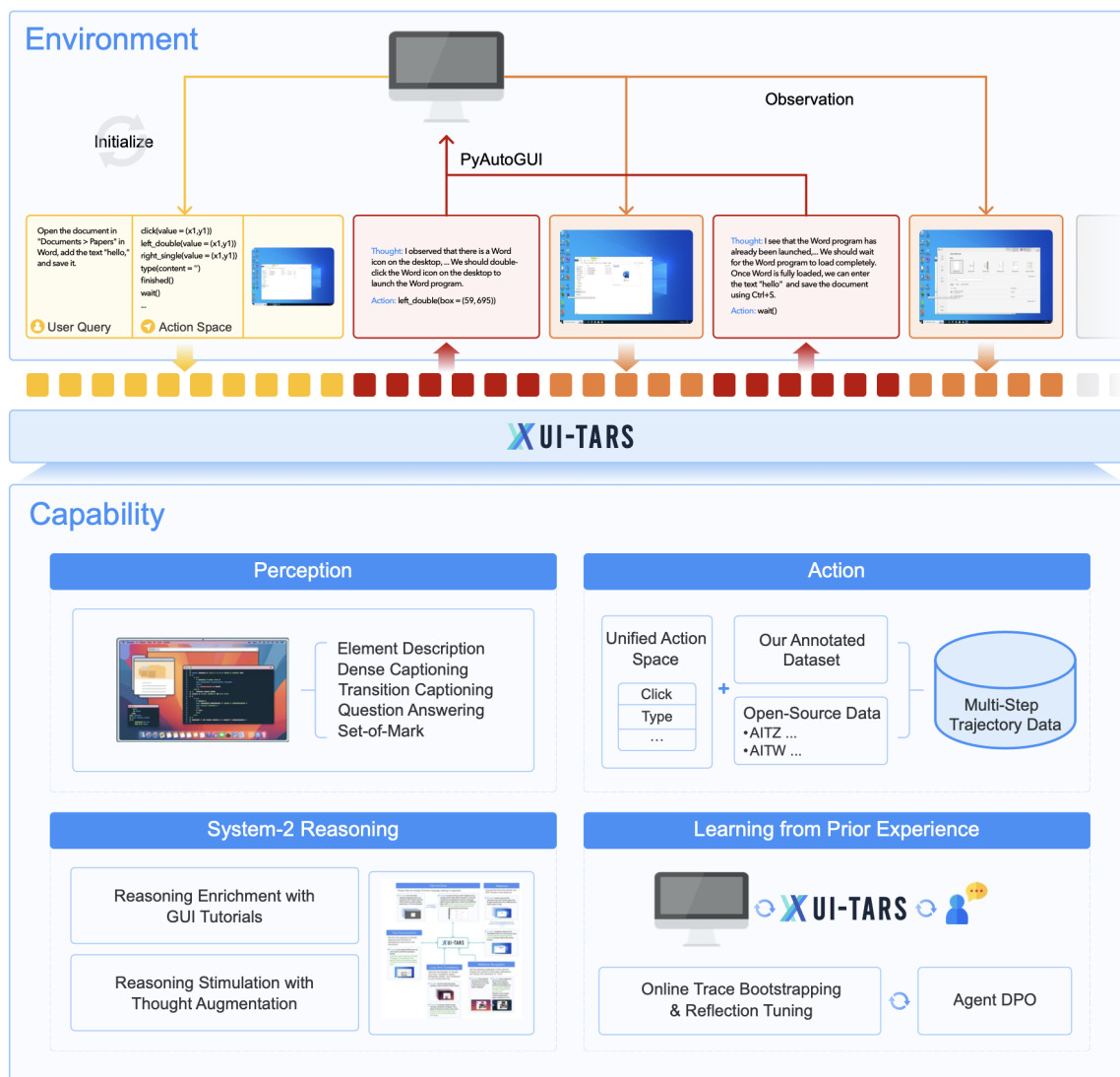


图 1: UI-TARS 总体架构（对应原文 Figure 4）。图中展示了模型架构及其四大核心能力。

这张图在说什么、数据怎么流动？整张图分三层：

- **顶层 Environment（环境）：** 左边是输入——**User Query（用户指令）**和**动作空间**。中间一台电脑表示 GUI 环境。数据流是一个闭环：模型输出动作经 **PyAutoGUI**（一个能模拟真实鼠标键盘的库）作用到环境上 → 环境返回新的 **Observation（截图观测）**。沿时间轴排开的一串小卡片就是 (o_i, t_i, a_i) 序列：每张卡片含一帧截图、一段 **Thought（思考，图中红字）** 和一个 **Action（动作）**——比如“我意识到打开了 Word，需要双击桌面图标……”然后执行 `left_double(...)`。这正对应 §4.1 的两条公式：动作前先想清楚。
- **中层 UI-TARS：** 就是那一个**端到端模型**本身，承接环境的输入、吐出思考与动作。强调“原生”

——不靠外挂模块。

- **底层 Capability (能力)**: 把全文方法的四块对应起来——**Perception (感知)** 列出五类任务 (元素描述/密集描述/转移描述/问答/SoM, §4.2); **Action (动作)** 展示统一动作空间 + 自标注数据 + 开源数据 → 多步轨迹数据 (§4.3); **System-2 Reasoning** 对应教程富化 + 思考增补 (§4.4); **Learning from Prior Experience (从先验经验学习)** 对应在线轨迹自举 + 反思微调 + Agent DPO (§4.5)。

一句话: 上面是“怎么和世界交互”(看截图 → 想 → 动 → 再看), 下面是“靠哪四种能力把这件事做好”, 中间是把这一切融成一体的那个模型。

4 实验解读 (§5)

模型与基线。三个规模: UI-TARS-2B / 7B / 72B (底座 Qwen-2-VL)。基线覆盖**商业模型** (GPT-4o、Claude-3.5-Sonnet、Gemini-1.5-Pro、Gemini-2.0 即 Project Mariner、Claude Computer-Use) 与**学术模型** (CogAgent、OmniParser、InternVL、Aria-UI、Aguvis、OS-Atlas、UGround、ShowUI、SeeClick、Qwen 系列等)。下文挑关键数字重排成小表。

4.1 感知能力 (§5.1)

三个基准: VisualWebBench (网页元素理解与定位)、WebSRC (网页结构理解 QA)、ScreenQA-short (移动屏幕内容理解 QA)。

表 3: GUI 感知基准结果 (节选自原文 Table 3, 数值越高越好; **粗体**为该列最佳)。

模型	VisualWebBench	WebSRC	ScreenQA-short
Claude-3.5-Sonnet	78.2	90.4	83.1
GPT-4o	78.5	87.7	82.3
UI-TARS-2B	72.9	89.2	86.4
UI-TARS-7B	79.7	93.6	87.7
UI-TARS-72B	82.8	89.3	88.6

说明什么: UI-TARS-72B 在 VisualWebBench 上 82.8 全面超过 GPT-4o (78.5) 和 Claude (78.2); 7B 在 WebSRC 上拿到 93.6 领先。**看界面看得准, 是后面能点对、做对任务的地基。**

4.2 定位能力 (§5.2)

三个基准: ScreenSpot Pro (高分辨率专业桌面环境, 23 个应用、5 个行业、3 种操作系统, **最难**)、ScreenSpot、ScreenSpot v2。

表 4: 定位能力关键结果 (综合自原文 Table 4-6, 报告平均分 Avg)。

模型	ScreenSpot Pro	ScreenSpot	ScreenSpot v2
OS-Atlas-7B	18.9	82.5	87.1
UGround-V1-7B	31.1	—	—
Claude Computer Use	17.1	83.0	—
Aguvis-72B	—	89.2	—
UI-TARS-2B	27.7	82.3	84.7
UI-TARS-7B	35.7	89.5	91.6
UI-TARS-72B	38.1	88.4	90.3

说明什么: 在最难的 ScreenSpot Pro 上, UI-TARS-72B 取得 38.1 的 SOTA, 远超 UGround-V1-7B (31.1) 和 OS-Atlas-7B (18.9)。一个有趣发现: **提高输入图像分辨率能在 ScreenSpot Pro 上显著涨点**; 而 ScreenSpot v1/v2 在 7B→72B 几乎不涨, 说明这两个老基准可能**不够难、难以充分体现大模型的定位上限**。

4.3 离线智能体能力 (§5.3)

三个基准: Multimodal Mind2Web (网页, 指标 Ele.Acc/Op.F1/Step SR)、AndroidControl (移动, 分 Low/High 两档)、GUI Odyssey (移动跨应用, 平均 15+ 步)。

关键结论: (1) 在 Mind2Web 上, **智能体模型普遍碾压框架方法** (用 GPT-4o/GPT-4V 当规划器的); UI-TARS-72B 多项指标 SOTA, 连 7B 都超过了 Aguvis-72B 和 Claude。(2) 在 AndroidControl 和 GUI Odyssey 上, UI-TARS-7B/72B 比前 SOTA (OS-Atlas-7B) **绝对提升约 25 个点**。(3) **Claude Computer-Use 在网页强、在手机上明显吃力**, 说明其 GUI 能力没很好迁移到移动端; 而 UI-TARS 网页和移动**两端都强**, 体现更好的适应性与泛化。

4.4 在线智能体能力 (§5.4)

两个动态交互基准: **OSWorld** (Ubuntu/Windows/macOS 桌面, 369 个真实任务, 仅截图模式, 3 次平均)、**AndroidWorld** (真实 Android 模拟器, 116 个任务、20 个应用)。

表 5: 在线基准结果 (节选自原文 Table 9)。OSWorld 为仅截图设置, 括号内为步数预算。

方法	OSWorld	AndroidWorld
GPT-4o (Agent Model)	5.0	34.5 (SoM)
GPT-4o + Aria-UI (框架)	15.2	44.8
Aguvis-72B	10.3	26.1
Claude Computer-Use	14.9 (15 步) / 22.0 (50 步)	27.9
UI-TARS-7B-DPO	18.7 (15 步)	33.0
UI-TARS-72B-SFT	18.8 (15 步)	46.6
UI-TARS-72B-DPO	22.7 (15 步) / 24.6 (50 步)	—

说明什么: (1) OSWorld 15 步预算下, UI-TARS-72B-DPO (22.7) 大幅超过 Claude (14.9); 而且 **UI-TARS 用 15 步 (22.7) 就追平了 Claude 用 50 步 (22.0)** ——执行效率高。50 步下 UI-TARS-72B-DPO 拿到 24.6 的新 SOTA, 超过所有框架方法。(2) AndroidWorld 上 UI-TARS-72B-SFT 得 46.6, 超过最强框架 (GPT-4o+Aria-UI, 44.8) 和最强模型 (Aguvis-72B, 26.1)。(3) **DPO 显著提升 OSWorld**, 证明“训练时引入负样本”能让模型更好区分优/次优动作。(4) **72B 在在线任务上比 7B 强很多, 且差距比离线大**——说明放大模型规模能显著提升 **System-2** 推理; 也说明光看离线基准会低估模型在真实动态环境里的能力。

4.5 System-1 vs. System-2 对比 (§5.5)

同一个 UI-TARS-7B 同时学会两种模式, 推理时用提示工程切换。用 **Best-of-N** ($N = 1, 16, 64$) 采样, 以 step 成功率衡量 (对应原文 Figure 8)。

两个反直觉但重要的发现:

- **域内 (in-domain):** 当 $N = 1$ (只采一次) 时, System-2 反而**略逊于** System-1——因为多步推理在单样本下可能引入无关或错误的推理步 (甚至指向不存在的对象), 增加幻觉风险, 模型容易“一条道走到黑”。但当 N 增到 16、64, **System-2 明显反超**: 候选多了, 就有机会跳出糟糕的推理路径, 正确动作落在采样里的概率大增。
- **域外 (OOD, AndroidWorld, 无对应训练数据):** 即便在 Bo1 ($N = 1$) 下, **System-2 也显著超过 System-1**。说明更深的推理在没见过的新任务上泛化优势明显。

核心洞见: System-2 在“多样性充足”时很强, 但要在**单次、果断输出**下也达到最优仍是挑战。理想方向是用**强化微调 (reinforced fine-tuning)** 让模型一次就高置信地给出正确动作——这也为后续 UI-TARS-1.5/2 的强化学习路线埋下伏笔。

5 核心 Takeaways 与对 AI 应用开发的启示

1. “一个端到端模型”打败“强模型 + 人手工拼的框架”。UI-TARS 证明：与其拿 GPT-4o 当大脑再外挂一堆提示词和工具模块，不如把感知/推理/记忆/动作**全部训进一个模型**。框架是设计驱动、脆弱难维护；原生模型是数据驱动、可持续进化。启示：做 GUI/桌面自动化产品时，长期看**数据驱动的专用模型**比堆叠提示工程更可扩展。
2. **纯视觉 + 统一动作空间 = 跨平台通吃**。只看截图、把 click/tap 等统一抽象、输出归一化坐标，让同一套能力在网页、桌面、手机间迁移。启示：设计自动化 agent 时，**统一的动作抽象层**能极大降低多端适配成本。
3. “先想后做” (System-2 thought) 确实有用，但要配合采样多样性。动作前显式生成思考能提升复杂/陌生任务表现；但单次采样时慢思考可能帮倒忙，需要 Best-of-N 或强化微调来兑现其潜力。启示：在产品里加 CoT 不是免费午餐，要权衡**延迟、采样次数与收益**。
4. 把“失败轨迹”当宝：**反思微调 + Agent DPO**。离线理想数据教不会模型纠错；让模型在真实环境里犯错、标注修正、甚至标注“错了之后怎么补救”，再用 DPO 同时利用正负样本。启示：**错误数据和成功数据一样有价值**，尤其对需要在线鲁棒性的 agent。
5. **在线自举突破数据瓶颈**。在数百台虚拟机上自动跑、规则 + VLM + 人工三级过滤、回灌迭代，是解决“GUI 过程数据稀缺”的工程范式。启示：**半自动数据飞轮**（采集 → 过滤 → 训练 → 再采集）是 agent 类产品冷启动后持续提升的可复制套路。
6. **规模化对“慢思考”的回报更大**；离线基准会低估真实能力。72B 在在线动态任务上对 7B 的优势远大于离线，且要在真实交互环境里才看得出来。启示：评估 agent **一定要上在线/真实环境**，别只看离线单步指标；推理密集型场景值得投更大模型。

参考文献（精选）

1. Yujia Qin, Yining Ye, Junjie Fang, et al. UI-TARS: Pioneering Automated GUI Interaction with Native Agents. arXiv preprint arXiv:2501.12326, 2025. <https://arxiv.org/abs/2501.12326>
2. Peng Wang, Shuai Bai, Sinan Tan, et al. Qwen2-VL: Enhancing Vision-Language Model's Perception of the World at Any Resolution. 2024.
3. Rafael Rafailov, Archit Sharma, Eric Mitchell, et al. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. NeurIPS, 2023.
4. Shunyu Yao, Jeffrey Zhao, Dian Yu, et al. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR, 2023.

5. Tianbao Xie, Danyang Zhang, Jixuan Chen, et al. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. NeurIPS, 2024.
6. Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, et al. AndroidWorld: A Dynamic Benchmarking Environment for Autonomous Agents. 2024.
7. Anthropic. Developing a Computer Use Model. 2024. <https://www.anthropic.com/news/developing-computer-use>